CONTROLE DE ELEVADORES VIA BUSCA EM ESPAÇO DE ESTADOS

Trabalho Prático de Busca — Inteligência Artificial

Professor: Jomi F. Hübner | Universidade Federal de Santa Catarina | 2026
Equipe: Kauan Abegg

Este relatório apresenta uma implementação executável do problema de controle de um elevador e compara estratégias clássicas de busca cega e informada. Os algoritmos clássicos constituem o núcleo do trabalho; o A* Direcional (A*D) é apresentado apenas como experimento complementar e não substitui BFS, A* ou busca gulosa. Os resultados apresentados foram produzidos pela execução do código Python incluído integralmente no Apêndice A deste PDF.

1. Problema

O prédio possui 50 andares. O elevador inicia vazio em um andar i, possui capacidade máxima c e recebe demandas no formato (origem, destino). Uma solução é uma sequência de operações UP, DOWN, IN e OUT que atende todas as demandas. **Neste trabalho, a qualidade é medida pelo número total de operações; portanto, menor custo significa melhor solução.**

2. Modelagem como espaço de estados

O estado é representado por **s = (f, I, W)**, em que f é o andar atual, I é o multiconjunto de destinos dos passageiros dentro do elevador e W é o multiconjunto das demandas que ainda aguardam. A capacidade c é parâmetro fixo da instância e, por isso, não precisa fazer parte do estado.

Operação	Pré-condição	Efeito
UP	$f < 50$	$f \leftarrow f + 1$
DOWN	$f > 1$	$f \leftarrow f - 1$
IN	Há demanda com origem f e $\|I\| < c$	Remove uma demanda de W e adiciona seu destino a I
OUT	Há passageiro em I com destino f	Remove um passageiro de I

Condição-meta: $I = \emptyset$ e $W = \emptyset$. Como cada ação tem custo unitário, g(n) é exatamente o número de operações realizadas até o nodo n.

3. Implementação

A implementação foi feita em Python 3, usando somente a biblioteca padrão. Cada estado é convertido em tuplas ordenadas, permitindo uso eficiente em dicionários e conjuntos. A busca evita reexpandir caminhos dominados e, no A*, mantém o melhor valor de g conhecido para cada estado.

Isso corresponde diretamente ao material da disciplina, no qual a busca é implementada a partir de estados, sucessores e uma estrutura de abertos. A própria página da disciplina disponibiliza uma implementação de busca em repositório e um notebook Python/Colab como material de apoio.

4. Estratégias de busca

Busca em Largura (BFS): Expande os nodos por ordem de profundidade. Como todas as operações custam 1, é completa e ótima.

A*: Escolhe o menor f(n)=g(n)+h(n). Com heurística admissível e o controle de melhor g usado na implementação, encontra solução ótima.

Busca Gulosa: Escolhe apenas pelo menor h(n). Tende a expandir poucos nodos, mas não garante solução ótima.

A* Direcional (A*D): Mantém f=g+h3 como critério principal e usa um desempate específico do elevador entre nodos com o mesmo f.

5. Heurísticas

Seja P o conjunto de andares que ainda precisam ser visitados: destinos de passageiros já embarcados e origens/destinos das demandas ainda esperando. As heurísticas testadas foram:

Heurística	Definição	Interpretação
h0	0	Referência sem informação.
h1	max |f − p|, p ∈ P	Distância até o andar relevante mais distante.
h2	(max P − min P) + |I| + |W|	Combina extensão vertical e parte das operações obrigatórias.
h3	Δ + δ + |I| + 2|W|	Inclui limite inferior de deslocamento e todas as operações IN/OUT inevitáveis.

Em h3, Δ = max(P) − min(P) e δ = min(|f − min(P)|, |f − max(P)|). O termo Δ+δ é o menor deslocamento necessário para cobrir o intervalo de andares relevantes se a ordem das visitas for relaxada. Além disso, cada passageiro dentro exige pelo menos um OUT e cada passageiro esperando exige pelo menos um IN e um OUT. Portanto, h3 não superestima o custo restante e é admissível.

Observação de escopo. A comparação dos algoritmos clássicos é mantida como parte principal do trabalho. A variação A*D aparece somente depois dessa comparação, como contribuição adicional.

6. Algoritmo proposto — A* Direcional (A*D)

O A*D é mantido apenas como extensão experimental do trabalho, depois da implementação e comparação dos algoritmos clássicos. Ele usa exatamente o mesmo valor principal do A* com h3. Quando dois estados possuem o mesmo f, o desempate prioriza: (1) estados em que é possível executar IN ou OUT no andar atual; (2) menor distância até um andar relevante; e (3) menor g. Assim, o conhecimento específico do problema só altera a ordem de expansão entre empates, sem substituir o A* como método principal.

Pseudocódigo:

```
OPEN ← fila de prioridade por (g+h3, atendimento, distância, g) inserir
estado inicial com g = 0 enquanto OPEN não estiver vazia:    n ← remover
primeiro estado    se n for meta: retornar caminho    para cada sucessor
válido s:
    se g(n)+1 melhorar o melhor g(s):          atualizar
pai e inserir s em OPEN
```

7. Experimentos

Foram usadas três instâncias totalmente especificadas. Os tempos abaixo são a mediana de 9 execuções em Python 3.13.5 no ambiente de referência; tempos podem variar entre computadores, enquanto custo e número de nós expandidos são determinísticos para esta implementação.

7.1 Instância Fácil andar inicial 1, capacidade 2, demandas [(2,5), (4,1)].

Algoritmo	Custo	Nós expandidos	Tempo mediano (ms)
BFS	12	76	0,0842
A* h0	12	76	0,1658
A* h1	12	38	0,1428
A* h2	12	37	0,1211
A* h3	12	21	0,2036
Gulosa h3	12	13	0,0501

A*D	12	20	0,1288

Na instância fácil, todas as estratégias encontraram custo 12. O A* h3 reduziu as expansões de 76 para 21, enquanto a gulosa encontrou por coincidência a mesma solução ótima com apenas 13 expansões.

7.2 Instância Média andar inicial 6, capacidade 3, demandas [(25,8), (6,14), (16,7), (13,2)].

Algoritmo	Custo	Nós expandidos	Tempo mediano (ms)
BFS	50	2231	2,7784
A* h0	50	2231	5,8803
A* h1	50	839	3,6243
A* h2	50	648	2,3565
A* h3	50	328	1,2894
Gulosa h3	67	99	0,3559
A*D	50	235	1,6344

Na instância média, BFS e A* encontraram custo ótimo 50. O A* h3 reduziu as expansões de 2.231 para 328 (aproximadamente 85,3%). A gulosa expandiu só 99 nodos, mas produziu custo 67.

7.3 Instância Difícil andar inicial 6, capacidade 3, demandas [(25,8), (6,14), (16,7), (13,2), (30,45)].

Algoritmo	Custo	Nós expandidos	Tempo mediano (ms)
BFS	92	10445	15,5746
A* h0	92	10445	35,5202
A* h1	92	5688	27,9205
A* h2	92	3283	14,3993
A* h3	92	1792	7,4941
Gulosa h3	123	228	0,8732
A*D	92	1370	11,6948

Na instância difícil, o custo ótimo foi 92. O A* h3 reduziu as expansões de 10.445 para 1.792 (aproximadamente 82,8%). A gulosa foi muito mais rápida, porém retornou 123 operações.

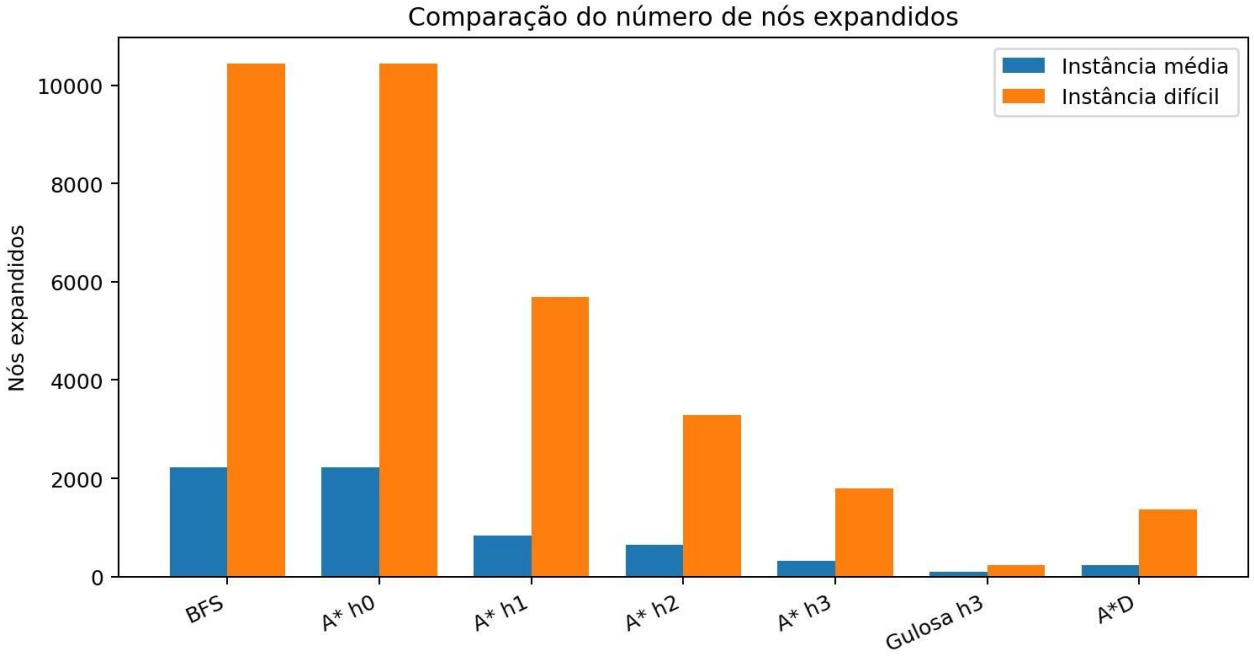


Figura 1 — Comparação do número de nós expandidos nas instâncias média e difícil.

8. Avaliação do A* Direcional

O desempate direcional preservou o mesmo custo do A* h3 nas três instâncias testadas e reduziu o número de expansões de 328 para 235 na instância média (28,4%) e de 1.792 para 1.370 na difícil (23,5%). Na instância fácil a diferença foi pequena: 21 contra 20 expansões. Isso indica que o desempate pode ser útil, mas o ganho depende da estrutura da instância e não deve ser generalizado como superior em todos os casos.

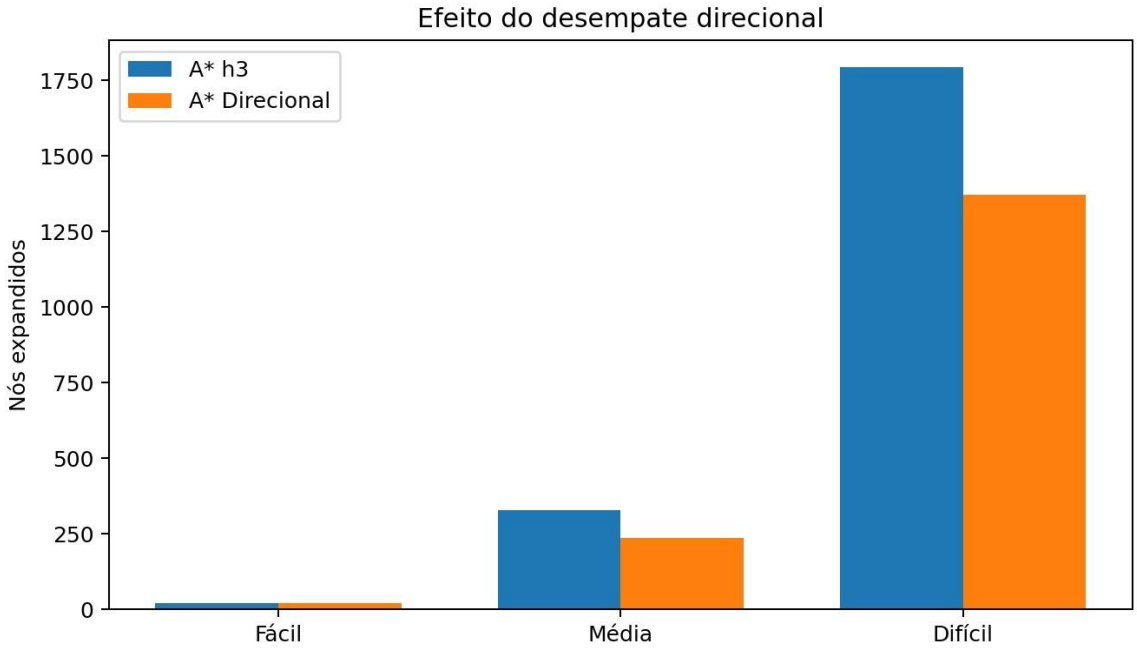


Figura 2 — Efeito do desempate do A Direcional sobre o número de nós expandidos.*

9. Exemplo de solução encontrada

Para a instância fácil, o A* h3 retornou o seguinte plano de custo 12:

UP → → → → → → IN(2 5) UP UP UP OUT(5) → DOWN → → → IN(4 1) DOWN → DOWN → DOWN → OUT(1)

10. Análise

Os experimentos mostram de forma clara o efeito da informação heurística. O A* h0 reproduz o comportamento da busca de custo uniforme e, com custos unitários, expande a mesma quantidade de estados que a BFS. À medida que a heurística incorpora mais informação, h1, h2 e principalmente h3 reduzem o espaço explorado.

A busca gulosa oferece o menor número de expansões, mas a instância média e a difícil demonstram sua limitação: ela pode aceitar um caminho globalmente pior porque ignora g(n). Para este problema, o A* h3 apresenta o melhor equilíbrio entre garantia de qualidade e redução do número de expansões.

11. Conclusão

O problema do elevador foi modelado como busca em espaço de estados e resolvido por diferentes estratégias. A implementação executável confirma que heurísticas mais informativas reduzem fortemente o número de nodos explorados. O A* com h3 manteve o custo ótimo em todas as instâncias e foi substancialmente mais eficiente que a BFS. A busca gulosa foi a mais econômica em expansões, porém perdeu qualidade nas instâncias maiores. O A* Direcional mostrou que um desempate específico do domínio pode reduzir expansões adicionais sem alterar o custo das soluções observadas.

12. Referências e materiais utilizados

- HÜBNER, Jomi F. Material da disciplina: Busca em Espaço de Estados. UFSC.
- Página da disciplina de Inteligência Artificial: https://jomifred.github.io/ia/
- Repositório de implementação de busca: https://github.com/jomifred/AI/tree/master/search
- Notebook Python indicado na página da disciplina: https://github.com/jomifred/AI/tree/master/search/python
- RUSSELL, Stuart; NORVIG, Peter. Artificial Intelligence: A Modern Approach.

APÊNDICE A - CÓDIGO-FONTE COMPLETO

A seguir está o código Python integral utilizado para gerar os experimentos apresentados neste relatório. Ele usa somente a biblioteca padrão do Python. Para executar: salve o conteúdo como elevadores_busca.py e rode `python elevadores_busca.py` em um terminal com Python 3 instalado.

```
#!/usr/bin/env python3
"""
Trabalho Pratico de Busca - Elevadores
UFSC - Inteligencia Artificial


Implementa:
- Busca em Largura (BFS)
- A* com h0, h1, h2 e h3
- Busca Gulosa com h3
- A* Direcional (A*D), usando h3 e desempate especifico do elevador


Nao usa bibliotecas externas.
"""


from collections import deque from
heapq import heappush, heappop from
time import perf_counter
import argparse
import csv import
itertools import
math import
statistics
 def estado(andar, dentro,
esperando):     return (
int(andar),
tuple(sorted(dentro)),
        tuple(sorted(tuple(x) for x in esperando)),
    )


 def
meta(s):
    _, dentro, esperando = s
    return len(dentro) == 0 and len(esperando) == 0
 def sucessores(s, capacidade):
andar, dentro, esperando = s
resposta = []
     if andar < 50:         resposta.append(("UP", (andar + 1,
dentro, esperando)))
     if andar > 1:         resposta.append(("DOWN", (andar - 1,
dentro, esperando)))


    # Se houver passageiro cujo destino e o andar atual, um deles pode sair.
    # Passageiros com o mesmo destino geram o mesmo estado sucessor.
if andar in dentro:         novo_dentro = list(dentro)
novo_dentro.remove(andar)
        resposta.append(
            (f"OUT({andar})", (andar, tuple(novo_dentro), esperando))
        )


    # Embarque de um passageiro esperando no andar atual.
    # Se houver demandas identicas, somente um sucessor distinto e necessario.
```

```
    if len(dentro) < capacidade:
destinos_ja_gerados = set()
         for indice, (origem, destino) in enumerate(esperando):
if origem == andar and destino not in destinos_ja_gerados:
destinos_ja_gerados.add(destino)


                novo_esperando = list(esperando)
novo_esperando.pop(indice)                  novo_dentro = list(dentro)
novo_dentro.append(destino)                  novo_dentro.sort()


                resposta.append(
                    (
                        f"IN({origem}->{destino})",
                        (
andar,
tuple(novo_dentro),
tuple(novo_esperando),
                        ),
                    )
                )
     return
resposta
 def posicoes_relevantes(s):
_, dentro, esperando = s
p = list(dentro)
     for origem, destino in
esperando:
        p.extend([origem, destino])
     return
p
 def h0(s):
return 0
 def h1(s):     andar, _, _ = s     p =
posicoes_relevantes(s)     return max((abs(andar - x)
for x in p), default=0)
 def
h2(s):
    _, dentro, esperando = s
p = posicoes_relevantes(s)


    deslocamento = (max(p) - min(p)) if p else 0
return deslocamento + len(dentro) + len(esperando)
 def h3(s):     andar, dentro,
esperando = s     p =
posicoes_relevantes(s)
     if
p:
        menor = min(p)
maior = max(p)
         delta = maior -
menor
        aproximacao = min(abs(andar - menor), abs(andar - maior))
        deslocamento = delta + aproximacao
else:         deslocamento = 0


    # Cada passageiro ja dentro obrigatoriamente precisa de OUT.
```

```
    # Cada passageiro esperando obrigatoriamente precisa de IN + OUT.
return deslocamento + len(dentro) + 2 * len(esperando)
 def prioridade_atendimento(s,
capacidade):     andar, dentro, esperando
= s
     pode_sair = andar in
dentro     pode_entrar = (
len(dentro) < capacidade
        and any(origem == andar for origem, _ in esperando)
    )


    # Menor valor = maior prioridade na heap.
return 0 if (pode_sair or pode_entrar) else 1
 def distancia_relevante_mais_proxima(s):     andar,
_, _ = s     p = posicoes_relevantes(s)     return
min((abs(andar - x) for x in p), default=0)
 def reconstruir(parent,
objetivo):
    caminho = []
atual = objetivo
     while parent[atual][0] is not
None:         anterior, acao =
parent[atual]
caminho.append(acao)         atual =
anterior

caminho.reverse()
return caminho
 def bfs(andar_inicial, capacidade, demandas):
inicial = estado(andar_inicial, (), demandas)
fila = deque([inicial])     parent = {inicial:
(None, None)}
     expandidos = 0
inicio = perf_counter()
     while fila:         atual
= fila.popleft()
         if meta(atual):             caminho =
reconstruir(parent, atual)
             return {
                 "custo": len(caminho),
                 "nos": expandidos,
                 "tempo_ms": (perf_counter() - inicio) * 1000,
                 "caminho": caminho,
             }
         expandidos +=
1
         for acao, prox in sucessores(atual,
capacidade):             if prox not in parent:
parent[prox] = (atual, acao)
fila.append(prox)
     return
None
 def
busca_prioridade(
andar_inicial,
capacidade,
demandas,
heuristica,
gulosa=False,
direcional=False,
):     inicial = estado(andar_inicial, (),
demandas)
```

```
    g = {inicial: 0}     parent =
{inicial: (None, None)}     fechados
= {}     contador =
itertools.count()     heap = []
    def prioridade(s, gs):
hs = heuristica(s)
        if gulosa:             return (hs,
gs, next(contador))
        f = gs +
hs
        if
direcional:
return (
f,
                prioridade_atendimento(s, capacidade),
distancia_relevante_mais_proxima(s),
                gs,
next(contador),
            )


        return (f, gs, next(contador))


    heappush(heap, (prioridade(inicial, 0), 0, inicial))
     expandidos = 0
inicio = perf_counter()
    while
heap:
        _, g_da_entrada, atual = heappop(heap)


        # Entrada antiga da heap, caso um caminho melhor tenha sido encontrado.
if g_da_entrada != g.get(atual):             continue
        if atual in fechados and fechados[atual] <=
g_da_entrada:             continue


        fechados[atual] = g_da_entrada
        if meta(atual):             caminho =
reconstruir(parent, atual)
            return {
                "custo": len(caminho),
                "nos": expandidos,
                "tempo_ms": (perf_counter() - inicio) * 1000,
                "caminho": caminho,
            }
        expandidos +=
1
        for acao, prox in sucessores(atual,
capacidade):             novo_g = g_da_entrada + 1
            if novo_g < g.get(prox,
math.inf):                 g[prox] = novo_g
parent[prox] = (atual, acao)
heappush(                    heap,
                    (prioridade(prox, novo_g), novo_g, prox),
                )
    return
None
```

```
INSTANCIAS = {
    "Facil": {
        "andar": 1,
        "capacidade": 2,
        "demandas": [(2, 5), (4, 1)],
    },
    "Media": {
        "andar": 6,
        "capacidade": 3,
        "demandas": [(25, 8), (6, 14), (16, 7), (13, 2)],
    },
    "Dificil": {
        "andar": 6,
        "capacidade": 3,
        "demandas": [
            (25, 8),
            (6, 14),
            (16, 7),
            (13, 2),
            (30, 45),
        ],
    },
}
```

```
ALGORITMOS = {
    "BFS": lambda i: bfs(
        i["andar"], i["capacidade"], i["demandas"]
    ),
    "A* h0": lambda i: busca_prioridade(
i["andar"], i["capacidade"], i["demandas"], h0
    ),
    "A* h1": lambda i: busca_prioridade(
i["andar"], i["capacidade"], i["demandas"], h1
    ),
    "A* h2": lambda i: busca_prioridade(
i["andar"], i["capacidade"], i["demandas"], h2
    ),
    "A* h3": lambda i: busca_prioridade(
i["andar"], i["capacidade"], i["demandas"], h3
    ),
    "Gulosa h3": lambda i: busca_prioridade(
        i["andar"],
i["capacidade"],
i["demandas"],
h3,          gulosa=True,
    ),
    "A*D": lambda i: busca_prioridade(
i["andar"],         i["capacidade"],
i["demandas"],         h3,
direcional=True,
    ),
}
```

```
 def
executar(repeticoes=7):
    linhas = []
caminhos = {}
     for nome_instancia, instancia in INSTANCIAS.items():
print(f"\n=== Instancia {nome_instancia} ===")
        print(
```

```
            "andar inicial =",
instancia["andar"],              "|
capacidade =",
instancia["capacidade"],
            "| demandas =",
instancia["demandas"],
        )
         for nome_algoritmo, funcao in
ALGORITMOS.items():
            execucoes = [funcao(instancia) for _ in range(repeticoes)]
            base = execucoes[0]              tempos =
[x["tempo_ms"] for x in execucoes]
tempo_mediano = statistics.median(tempos)


            linhas.append(
                {
                    "instancia": nome_instancia,
                    "algoritmo": nome_algoritmo,
                    "custo": base["custo"],
                    "nos_expandidos": base["nos"],
                    "tempo_mediano_ms": round(tempo_mediano, 4),
                }
            )


            caminhos[(nome_instancia,  nome_algoritmo)]  =  base["caminho"]
print(                                        f"{nome_algoritmo:10s}   |   "
f"custo={base['custo']:3d} | "                   f"nos={base['nos']:6d} | "
f"tempo mediano={tempo_mediano:8.4f} ms"
            )
     return linhas,
caminhos
 def salvar_csv(linhas, arquivo):     with open(arquivo, "w",
newline="", encoding="utf-8") as f:         writer =
csv.DictWriter(             f,             fieldnames=[
"instancia",
                "algoritmo",
                "custo",
                "nos_expandidos",
                "tempo_mediano_ms",
            ],
        )
writer.writeheader()
writer.writerows(linhas)
 def main():     parser =
argparse.ArgumentParser()
parser.add_argument(         "--
repeticoes",         type=int,
default=7,
        help="numero de execucoes usadas para calcular o tempo mediano",
    )
parser.add_argument(
        "--csv",
default="resultados_elevadores.csv",
help="arquivo CSV de saida",
    )     args =
parser.parse_args()


    linhas, caminhos = executar(max(1, args.repeticoes))
salvar_csv(linhas, args.csv)
```

```
    print("\nCSV salvo em:", args.csv)


    print("\nExemplo de plano encontrado para a instancia Facil (A* h3):")
print(" -> ".join(caminhos[("Facil", "A* h3")]))
 if __name__ ==
"__main__":     main()
```

APÊNDICE B - SAÍDA DE EXECUÇÃO DE REFERÊNCIA

Trecho da execução usada para validar o código e conferir os valores apresentados nas tabelas. Os tempos podem variar entre computadores; os custos e a contagem de nós desta implementação são reproduzíveis.

```
=== Instancia Facil ===
andar inicial = 1 | capacidade = 2 | demandas = [(2, 5), (4, 1)]
BFS        | custo= 12 | nos=    76 | tempo mediano=  0.0842 ms
A* h0      | custo= 12 | nos=    76 | tempo mediano=  0.1658 ms
A* h1      | custo= 12 | nos=    38 | tempo mediano=  0.1428 ms
A* h2      | custo= 12 | nos=    37 | tempo mediano=  0.1211 ms
A* h3      | custo= 12 | nos=    21 | tempo mediano=  0.2036 ms
Gulosa h3  | custo= 12 | nos=    13 | tempo mediano=  0.0501 ms
A*D        | custo= 12 | nos=    20 | tempo mediano=  0.1288 ms

=== Instancia Media ===
andar inicial = 6 | capacidade = 3 | demandas = [(25, 8), (6, 14), (16, 7), (13, 2)]
BFS        | custo= 50 | nos=  2231 | tempo mediano=  2.7784 ms
A* h0      | custo= 50 | nos=  2231 | tempo mediano=  5.8803 ms
A* h1      | custo= 50 | nos=   839 | tempo mediano=  3.6243 ms
A* h2      | custo= 50 | nos=   648 | tempo mediano=  2.3565 ms
A* h3      | custo= 50 | nos=   328 | tempo mediano=  1.2894 ms
Gulosa h3  | custo= 67 | nos=    99 | tempo mediano=  0.3559 ms
A*D        | custo= 50 | nos=   235 | tempo mediano=  1.6344 ms

=== Instancia Dificil ===
andar inicial = 6 | capacidade = 3 | demandas = [(25, 8), (6, 14), (16, 7), (13, 2), (30, 45)]
BFS        | custo= 92 | nos= 10445 | tempo mediano= 15.5746 ms
A* h0      | custo= 92 | nos= 10445 | tempo mediano= 35.5202 ms
A* h1      | custo= 92 | nos=  5688 | tempo mediano= 27.9205 ms
A* h2      | custo= 92 | nos=  3283 | tempo mediano= 14.3993 ms
A* h3      | custo= 92 | nos=  1792 | tempo mediano=  7.4941 ms
Gulosa h3  | custo=123 | nos=   228 | tempo mediano=  0.8732 ms
A*D        | custo= 92 | nos=  1370 | tempo mediano= 11.6948 ms
```

CSV salvo em: resultados_elevadores.csv

```
Exemplo de plano encontrado para a instancia Facil (A* h3):
UP -> IN(2->5) -> UP -> UP -> UP -> OUT(5) -> DOWN -> IN(4->1) -> DOWN -> DOWN -> DOWN -> OUT(1)
```